Readme file:

Task1:

Install all the required python dependencies:(you can use requirements1.py)

Libraries used:

langchain

langchain-google-genai

langchain-community

python-dotenv

tavily-python

Note: There should be .env file where apis for gemini and tavily-python will be stored as "GOOGLE_API_KEY" and "TAVILY_API_KEY"

Folder Structure

├ task1.py # Main script to generate the labeled dataset

├ tools_new.py # Contains search and LLM summarization logic

├ llm.py # LLM initialization using Google Gemini

├ requirements.txt # List of all required Python packages

├.env # API keys (GOOGLE_API_KEY, TAVILY_API_KEY)

├ labeled_dataset2.0.json # Output JSON file with summaries and categories

├ labeled_dataset1.5.json # Output JSON file with summaries and categories

Use: python task1.py

Final submitted files from task1:

- 50 labeled, structured and documented dataset from each 1.5 model and 2.0 model as "labeled_dataset2.0.json" and "labeled_dataset1.5.json" .
- Code to create the initial dataset which can extract articles from web, summarize them and make a json file.(task1.py)
- One raw created dataset from task1 named as "labelled_dataset1.5.json"

Features Implemented

`task1.py`

- Loops through 50 pre-defined cyber incident queries.

- Uses a search tool to find relevant online articles.

- Summarizes each article using Gemini LLM.

- Classifies each incident into one or more of 8 custom categories.

- Saves results in `labeled_datasetX.json`.

`tools_new.py`

- Uses LangChain and Tavily to search the web.

- Summarizes the top search result using Gemini.

- Applies a classification prompt to map incident summaries to categories.

- Includes light post-processing and fallback logic for robustness.

`llm.py`

- Sets up Gemini LLM (`gemini-2.0-flash` or `gemini-1.5-flash`) with low temperature for factual generation.

Note: After getting dataset of both models(after switching one after another in llm.py code), we did human validation part and labelled category_flow from my observation, then combined human validated category_flow and llm_suggested category flow in a single dataset for each model, and named them :

|- final_dataset_g2.0.json #the self validated final dataset of dataset created in task1 of 2.0 output

|- final_dataset_g1.5.json #the self validated final dataset of dataset created in task1 of 1.5 output

Output and screenshots are shown in Task1 report.

Task2:

Install all the required python dependencies:(you can use requirements1.py and requirements2.py)

Libraries used:

pandas

numpy

scikit-learn

matplotlib

seaborn

langchain

langchain-google-genai

langchain-community

python-dotenv

Folder Structure:

|-task2.py # Main script

|-.env # Contains API key

|-Model_evaluation.ipynb #do model analysis

|- final_dataset_g2.0.json #the self validated final dataset of dataset created in task1 of 2.0 output

|- final_dataset_g1.5.json #the self validated final dataset of dataset created in task1 of 1.5 output

|-attack_incident_updated.csv # created during model evaluation

Use: python task2.py

Final submitted files from task2

- **Model_evaluation.ipynb file for model evaluation matrix and also to create csv file for further data analysis.**
- Attack_incident_updated.csv file created by model_evaluation.pynb
- Task2.py file to receives an incident description and get category flow output.

Output and screenshots are shown in Task2 report.

Bonus Task:

Install all the required python dependencies: (you can use requirements3.py)

Libraries used:

streamlit

pandas

numpy

scikit-learn

altair

matplotlib

seaborn

langchain

google-generativeai

python-dotenv

regex

note: You should have a .env file where gemini api is stored as “GOOGLE_API_KEY” variable.

Use: streamlit run bonus_task.py

Required folders:

├ bonus_task.py #streamlit file

├ .env # your Google API key

├attack_incident_updated.csv # the dataset used in the app generated by model_evaluation.py

├Annotation_tool_output.csv # created after annotations

Final submition:

- Bonus_task.py
- Annotation_tool_output.csv (if user do some new annotation)

Output and screenshots are shown in bonus_task report.